

# Discussion 9

**Note:** Your TA will probably not cover all the problems on this worksheet. The discussion worksheets are not designed to be finished within an hour. They are deliberately made slightly longer so they can serve as resources you can use to practice, reinforce, and build upon concepts discussed in lectures, discussions, and homework.

## This Week's Cool AI Demo/Video:

Extropic TSU: <https://www.youtube.com/watch?v=Y28JQzS6TlE>

## 1 Convolution Operation in CNNs

In this question, we will look at the mechanism of weight sharing in convolutions. Suppose that we have an input vector with  $D = 9$  dimensions:

$$\mathbf{x} = [x_1 \quad x_2 \quad \dots \quad x_9]^T.$$

We compute a 1D convolution with the kernel filter that has  $K = 3$  weights (parameters):

$$\mathbf{k} = [k_1 \quad k_2 \quad k_3]^T.$$

- (a) What's the output dimension if we apply filter  $\mathbf{k}$  with no padding and stride of 1? What's the first element of the output? What's the last element?

### Solution:

The output dimension is  $9 - 3 + 1 = 7$ . The first element is  $k_1x_1 + k_2x_2 + k_3x_3$  and the last element is  $k_1x_7 + k_2x_8 + k_3x_9$ . Expressed as a dot product, these are equivalent to  $[x_1 \quad x_2 \quad x_3]\mathbf{k}$  and  $[x_7 \quad x_8 \quad x_9]\mathbf{k}$ .

- (b) What's the output dimension if we apply filter  $\mathbf{k}$  with padding of size 1 and stride of 2? What's the first element of the output? What's the second element?

### Solution:

For the second example, the output dimension would be  $(9 + 2 - 3)/2 + 1 = 5$ . The first element is  $k_1 \cdot 0 + k_2x_1 + k_3x_2$  and the second element would be  $k_1x_2 + k_2x_3 + k_3x_4$ . Expressed as a dot product, these are equivalent to  $[0 \quad x_1 \quad x_2]\mathbf{k}$  and  $[x_2 \quad x_3 \quad x_4]\mathbf{k}$ .

- (c) Recall that CNN filters have the property of weight sharing, meaning that different portions of an image can share the same weight to extract the same set of features. A convolution is actually a linear operation, which we can express as  $\mathbf{x}' = \mathbf{K}\mathbf{x}$  for some weight matrix  $\mathbf{K}$ . Find  $\mathbf{K}$  for the convolution applied in part (a). What is its dimension?

**Solution:**

The input size is  $D = 9$  and the output size shrinks by  $K - 1 = 2$ , so  $\mathbf{K}$  has a dimension of  $\mathbb{R}^{7 \times 9}$  and is given by

$$\mathbf{K} = \begin{bmatrix} k_1 & k_2 & k_3 & 0 & 0 & \cdots & 0 \\ 0 & k_1 & k_2 & k_3 & 0 & \cdots & 0 \\ & & & \vdots & & & \\ 0 & \cdots & 0 & k_1 & k_2 & k_3 & 0 \\ 0 & \cdots & 0 & 0 & k_1 & k_2 & k_3 \end{bmatrix}.$$

- (d) Consider now a 2-dimensional example with the following kernel filter and image:

$$\mathbf{K} = \frac{1}{4} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad \mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Using no padding and a stride of 1, compute the output, and describe the effect of this filter.

**Solution:**

The output is

$$\begin{bmatrix} 3 & 4 \\ 6 & 7 \end{bmatrix}.$$

This is a 2D moving average filter (aka a box filter) and can be used to perform average pooling.

- (e) We want to know the general formula for computing the output dimension of a convolution operation. Suppose we have a square input image of dimension  $W \times W$  and a  $K \times K$  kernel filter. If we assume stride of 1 and no padding, what's the output dimension  $W'$ ? What if we applied stride of  $s$  and padding of size  $p$ , how would the dimension change?

**Solution:**

With a stride length of one and no padding, the output dimension will be  $W' = W - K + 1$ .

With a stride of  $s$  and padding of size  $p$ , the output dimension will be  $\lfloor \frac{W+2p-K}{s} \rfloor + 1$ . We include a  $+2p$  because the padding is equivalent to adding extra terms on both the left and right side of the array. Dividing by  $s$  follows from the fact that we increment our counter by  $+s$  instead of  $+1$  when sliding along the image. The floor division operator ensures that the resulting output is an integer.

## 2 L2 Regularization & Weight Decay

Derive the stochastic gradient descent (SGD) update rule when we add an  $L_2$  regularization term  $\lambda\|\mathbf{w}\|^2$  to a error function  $E(\mathbf{w})$ , and explain why we refer to this as weight decay regularization.

**Solution:**

The regularized objective is

$$E_{\text{reg}}(\mathbf{w}) = E(\mathbf{w}) + \lambda\|\mathbf{w}\|^2$$

and its gradient is

$$\nabla E_{\text{reg}}(\mathbf{w}) = \nabla E(\mathbf{w}) + 2\lambda\mathbf{w}.$$

The SGD update rule is

$$\begin{aligned}\mathbf{w} &\leftarrow \mathbf{w} - \epsilon \nabla E_{\text{reg}}(\mathbf{w}) \\ &= \mathbf{w} - \epsilon (\nabla E(\mathbf{w}) + 2\lambda\mathbf{w}) \\ &= (1 - 2\epsilon\lambda)\mathbf{w} - \epsilon \nabla E(\mathbf{w}).\end{aligned}$$

This update rule shows that the weight vector is shrunk by a factor of  $(1 - 2\epsilon\lambda)$  on each update, as well as gaining a contribution  $-\epsilon\nabla E$ . That is, every weight decays slightly towards zero on each step if not prevented from doing so by opposition from the gradient of  $E$ .

### 3 Vanishing Gradients & ResNets

The deeper a neural network grows, the more likely it will suffer from vanishing gradients because downstream layers (i.e., earlier layers) are the product of many backpropagated gradients—any of which may be close to zero. This question explores one approach to address vanishing gradients.

- (a) The vanishing gradient problem can arise from saturating activation functions, such as the sigmoid,  $s(x) = \frac{1}{1+e^{-x}}$ . What's the maximum gradient of the sigmoid function? Why might ReLU be more preferable than the sigmoid function in deep networks?

**Solution:**

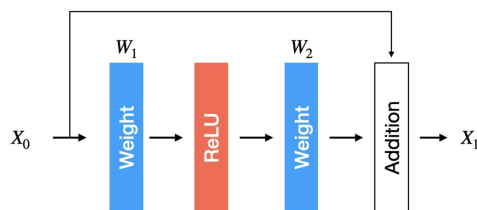
We start by computing the derivative of the sigmoid function:

$$\begin{aligned}\frac{d}{dx}s(x) &= \frac{d}{dx}\left(\frac{1}{1+e^{-x}}\right) \\ &= \frac{1}{(1+e^{-x})^2} \cdot e^{-x} \\ &= \frac{1}{(1+e^{-x})} \cdot \frac{e^{-x}}{(1+e^{-x})} \\ &= s(x) \cdot (1-s(x))\end{aligned}$$

The output of the sigmoid function falls in the range of  $(0, 1)$ . The maximum gradient of the sigmoid is thus achieved when  $s(x) = (1-s(x)) = 1/2$  and the maximum gradient is  $\frac{1}{4}$ . If we multiple many of these gradients together, they will decay towards zero exponentially.

In contrast, the maximum gradient of ReLU is 1, making vanishing gradients less problematic. (Instead, parameters can get stuck at zero, which is referred as the “dying ReLU” problem.)

- (b) One common way to tackle vanishing gradients is by adding residual connections. The figure below shows a simplified example of a residual block. Assuming that the weights consist of an affine layer with zero biases, write an expression for  $X_1$  in terms of  $W_1$ ,  $W_2$  and  $X_0$ .



**Solution:**

$$X_1 = W_2 \text{ReLU}(W_1 X_0) + X_0$$

- (c) Compute the gradient  $\frac{\partial X_1}{\partial X_0}$ . Based on what you see numerically, why does a residual connection preserve gradient norms better? (You may assume  $X_0$  and  $X_1$  are scalar.)

**Solution:**

The ReLU function is not differentiable at the origin, so we express the gradient case-wise:

$$\frac{\partial X_1}{\partial X_0} = \begin{cases} W_2 W_1 + 1 & \text{if } W_1 X_0 > 0 \\ 1 & \text{otherwise} \end{cases}$$

By enumerating the possible values, we see that there's always an identity term which helps preserve the norm of the gradient, thus avoiding the vanishing gradient problem.

## 4 Dropout

In this question we explore dropout as a form of regularization. Recall that when we apply dropout during training, we randomly turn off individual nodes, as shown below.

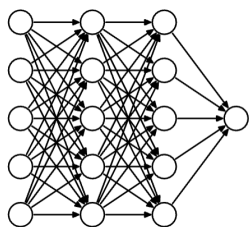


Figure 1: Original (and Final) Net

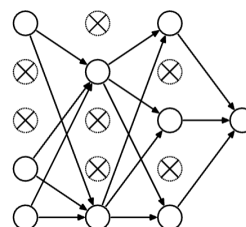


Figure 2: Net during Dropout Training

- (a) Explain qualitatively how dropout regularization works and why we can view dropout as simulating an ensemble of many “sub-networks.” Describe what happens during training when dropout is applied to a neural network’s neurons.

**Solution:**

Dropout can be seen as simulating an ensemble of networks. Each training example updates a randomly chosen subnetwork. The learned weights are shared among all these subnetworks (they’re just different subsets of the same global weights). At test, we approximate averaging over the ensemble by using the full network with appropriately scaled weights. This averaging effect typically improves generalization, much like model ensembling does, but in a far more computationally efficient manner than training many separate networks from scratch.

- (b) Dropout has a tendency to prevent any one weight from growing excessively large, compared to other weights in the network. Why would that be?

**Solution:**

If a weight is very large, it has a large influence: a small change to the unit activation feeding the weight can cause a large change in the units downstream from that edge. So when the weight is turned off by dropout, it is likely to cause large changes to some or all outputs. Since dropout sometimes trains with the weight on and sometimes trains with the weight off, the network is learning to make correct predictions in both circumstances, so the weight cannot grow too large. Instead, many small weights share the burden of voting for a correct answer.

- (c) Consider a ReLU layer that computes  $\mathbf{z} = h(\mathbf{W}\mathbf{x})$ , where  $\mathbf{x}$  represents the nodes in some layer,  $\mathbf{z}$  represents the nodes in the subsequent layer,  $\mathbf{W}$  is the weight matrix connecting those two layers, and  $h(\cdot)$  is the ReLU activation function applied element-wise to a vector. Recall that the backpropagation step updates the weights in  $\mathbf{W}$  by computing

$$\nabla_{\mathbf{w}_i} L = \frac{\partial L}{\partial z_i} h'(\mathbf{w}_i \cdot \mathbf{x}) \mathbf{x} \quad \text{and then} \quad \mathbf{w}_i \leftarrow \mathbf{w}_i - \eta \nabla_{\mathbf{w}_i} L,$$

where  $L$  is the loss function for a training point,  $\mathbf{w}_i$  is the  $i$ th row of  $\mathbf{W}$ , and  $\eta$  is the learning rate/step size. If dropout turns off some fraction of the units in both layers of hidden units, how should we implement this backpropagation step?

**Solution:**

First, if unit  $z_i$  is turned off, don't update  $\mathbf{w}_i$  at all.

If unit  $x_d$  is turned off, set the corresponding component of  $\mathbf{x}$  to zero before computing  $\nabla_{\mathbf{w}_i} L$ .

- (d) Consider a single neuron receiving some input during training with dropout. Derive an expression for the expected output of this neuron (pre-activation) under dropout. (Assume a fixed input and weight configuration, and that each unit is dropped independently with a given probability. You may assume a “keep probability”  $p$  for each unit – i.e. each neuron is retained with probability  $p$  during training.) Show how this relates to the test-time behavior of scaling weights or activations.

**Solution:**

Ignoring the activation function for now, consider a neuron that computes:

$$\mathbf{z} = w_1x_1 + w_2x_2 + \cdots + w_Dx_D + w_0,$$

where  $x_d$  are inputs from the previous layer and  $w_d$  are the corresponding weights.

While training with dropout, each incoming neuron  $x_d$  might be dropped. For simplicity, assume each neuron  $x_d$  is kept with probability  $p$  (and dropped with probability  $1 - p$ ) independently. Introduce random Bernoulli mask variables  $m_d$  for each input, where  $m_d = 1$  with probability  $p$  (meaning input  $x_d$  is retained) and  $m_d = 0$  with probability  $1 - p$  (input dropped, effectively  $x_d$  replaced by 0). The neuron's pre-activation output under dropout is:

$$\mathbf{z}_{\text{drop}} = \sum_{d=1}^D w_d(m_dx_d) + w_0$$

Because  $m_d$  are random,  $\mathbf{z}_{\text{drop}}$  is also random (during training). We can compute its expectation (with respect to the dropout randomness  $m_d$ ) given the fixed input values and current weights:

$$\mathbb{E}[\mathbf{z}_{\text{drop}}] = \mathbb{E} \left[ \sum_{d=1}^D w_d(m_dx_d) + w_0 \right].$$

Using the linearity of expectation and the fact that only  $m_d$  is random,

$$\mathbb{E}[\mathbf{z}_{\text{drop}}] = \sum_{d=1}^D w_d \mathbb{E}[m_d] x_d + w_0$$

Because  $m_d$  is a Bernoulli random variable with probability of success  $p$ ,  $\mathbb{E}[m_d] = p$ . Thus,

$$\mathbb{E}[\mathbf{z}_{\text{drop}}] = \sum_{d=1}^D w_dp x_d + w_0 = p \sum_{d=1}^D w_dx_d + w_0 = p\mathbf{z} + (1 - p)w_0.$$

Now we can see that dropout scales the expected output by the keep probability  $p$ . This is why, at test time, we adjust the network to match the average scale seen during training by multiplying outputs by  $p$ .